

UNIVERSITY OF WOLLONGONG IN DUBAI

Accessibility Scene Hazard Assistant

An Integrated Computer Vision Application for Accessible Scene Guidance

CSCI435 - Computer Vision Algorithms and Systems
Spring 2026

Mehdi Leghmizi | 8528834

Neeraj Santosh | 8329345

Muhammad Soban | 8555588

Zachary Bracke | 8947405

Mostafa Shalash | 7391493

Lecturer: Dr Patrick Mukala

Report date: 27 June 2026

Report at a glance

We developed a locally deployable web application that combines image enhancement, edge detection, object proposal, custom-trained object recognition, moving-object detection and object tracking in one workflow. The system accepts an uploaded image, webcam capture or video and returns annotated visual results, text guidance, latency and estimated throughput.

Key outcome. The executed notebook measured 92.5% mean precision, 88.1% recall and 89.8% F1 across seven scene conditions. The full still-frame pipeline averaged 74.2 FPS (13.5 ms), exceeding the 10 FPS requirement.

Abstract

This project investigates how several computer vision techniques can operate together as a usable system rather than as isolated laboratory exercises. The proof-of-concept is an accessibility-oriented scene assistant for recognising three marker types: stop, warning and safe route. A deterministic custom dataset of 560 augmented marker crops is generated inside the notebook. HOG, HSV, intensity and Hu-moment features are used to compare an RBF support vector machine, random forest and k-nearest-neighbour classifier. The selected SVM is then integrated with colour/contour proposals, adaptive Canny edges, CLAHE enhancement, MOG2 background modelling and centroid tracking. Evaluation covers held-out recognition, class-aware scene detection, lighting and orientation changes, noise, blur, occlusion, clutter, blank inputs and CPU speed. Results are strong within the controlled synthetic domain, but the report also identifies the synthetic-to-real gap and the need for user testing before any real assistive deployment.

Submission components

- Self-contained and executed Jupyter notebook: CSCI435_Project.ipynb.
- Dependency file and reproducible execution instructions in README.md.
- Two-minute-five-second demonstration video generated from actual pipeline outputs.
- Editable DOCX report and submission-ready PDF report.
- Timed defence guide, robust sample inputs and requirement-to-evidence matrix.

Introduction

Problem statement

People with limited vision may need a quick indication of warnings, safe-route cues, visible boundaries and movement in a camera view. Many classroom demonstrations process one image with one algorithm, which does not show how a usable application should validate input, combine results or communicate uncertainty. Our goal was therefore to create one deployable interface that links low-level, mid-level and high-level vision tasks around a clear accessibility user story.

User story

User story. As a user, I want to upload an image or video, or capture a webcam image, so that one interface enhances the scene, recognises marker-like objects, highlights boundaries and movement, and gives concise guidance.

Selected vision capabilities

- Image enhancement: gray-world colour balancing followed by CLAHE on the luminance channel.
- Edge detection: adaptive Canny thresholds with morphological closing and a visible edge overlay.
- Object detection: HSV colour segmentation, morphology, contour filtering and bounding boxes.
- Object recognition: an RBF SVM trained on the group's generated custom marker dataset.
- Change and moving-object detection: MOG2 adaptive background modelling on video frames.
- Object tracking: centroid association with persistent IDs and missing-frame tolerance.

Scope and safety

The application is an academic proof-of-concept, not a certified safety device. Its marker vocabulary and synthetic evaluation are deliberately constrained so that every training and testing step is reproducible. Real-world assistive use would require representative data, calibrated depth, accessibility studies, privacy review and a much more conservative false-negative analysis.

System architecture

Accessibility Scene Hazard Assistant - System Architecture

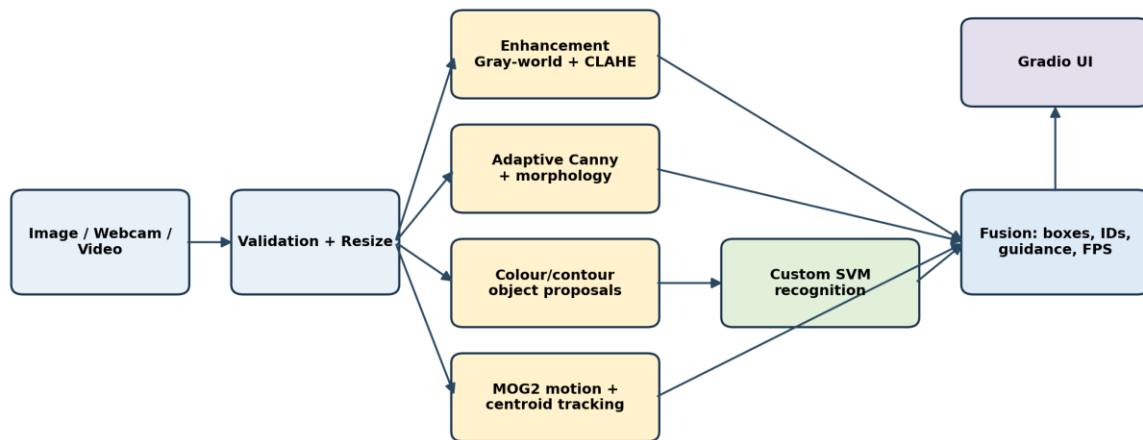


Figure 1. System architecture and data flow.

The frontend and backend are packaged in the notebook. Gradio provides the browser interface while Python, OpenCV and scikit-learn provide the processing and model layers. An input is validated and resized to a maximum width of 640 pixels. The enhanced frame feeds edge and colour-proposal branches. Candidate pixels are canonicalised and classified using the custom SVM. Video frames also pass through MOG2 and the centroid tracker. Fusion draws boxes, labels and track IDs, builds a text guidance sentence and reports latency/FPS.

Frontend and input modalities

The Image / Webcam tab accepts an upload or camera capture and shows four outputs: the integrated annotated result, enhanced image, edge view and guidance. The Video tab accepts an uploaded file, applies the same frame pipeline plus motion/tracking, and returns an annotated video and aggregate metrics. These two paths satisfy the requirement for at least two input modalities while keeping the interface consistent.

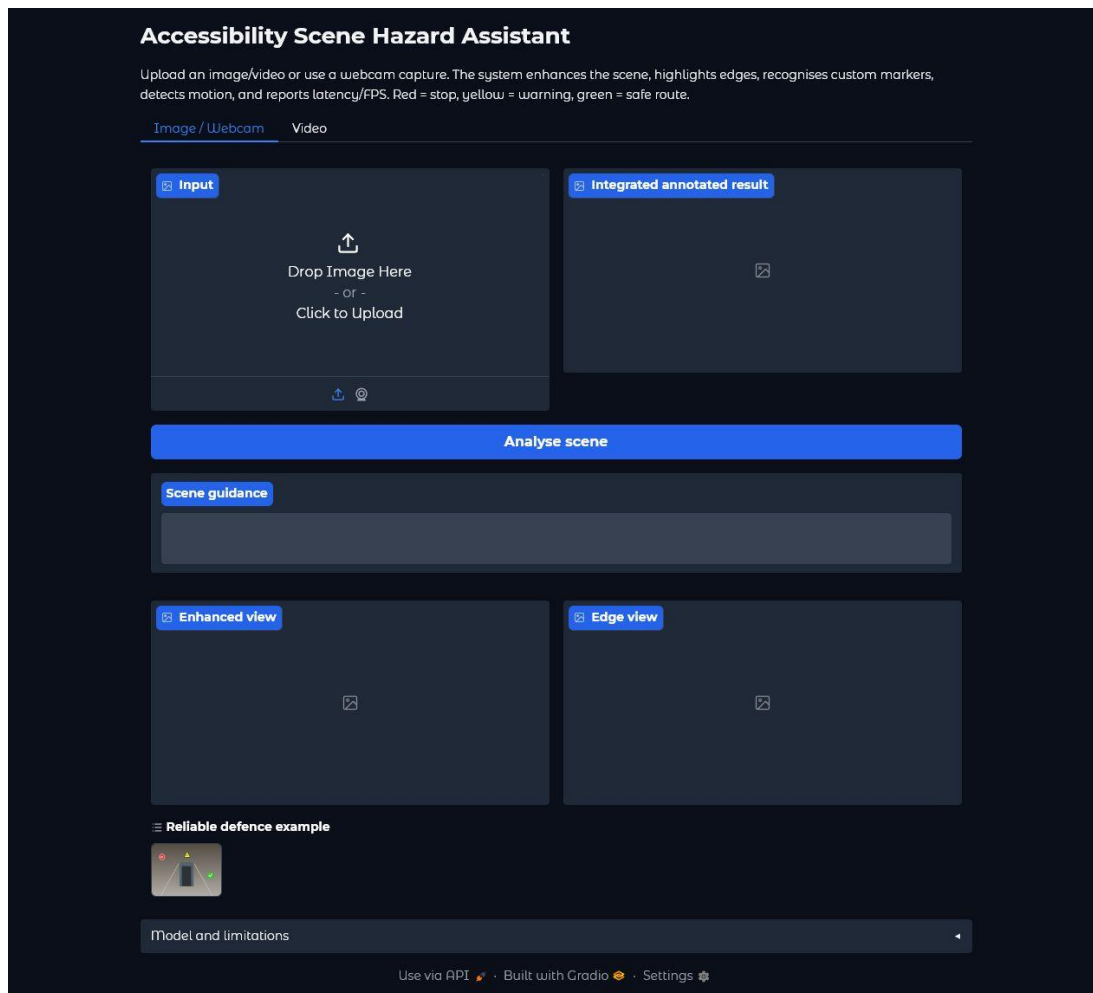


Figure 2. Verified local Gradio interface in the Image / Webcam mode.

Implementation details

Enhancement and edge processing

Gray-world balancing estimates a target mean across BGR channels and limits channel scaling to avoid extreme colour shifts. The result is converted to CIELAB, where CLAHE is applied only to luminance. Adaptive Canny thresholds are calculated from the median of a Gaussian-smoothed grayscale image; a 3 x 3 closing operation joins short breaks. This branch supplies a clear boundary view but does not directly claim semantic meaning.

Object proposal and custom recognition

HSV thresholds form red, yellow and green proposal masks. Opening removes isolated pixels and closing joins small gaps. Contours below 300 pixels or outside broad aspect-ratio limits are rejected. Each remaining object is centred on a neutral 96 x 96 canvas at a fixed foreground scale. This canonicalisation solved an important training/inference mismatch: the recogniser now sees the same object scale during

both phases. Proposal masks come from the enhanced frame, while recognition uses pre-enhancement validated pixels to preserve the training appearance distribution.

```
enhanced = enhance_image(frame)
edges = adaptive_canny(enhanced)
detections = detect_markers(
    enhanced, self.model, classification_frame=frame
)
```

Motion detection and tracking

For video, OpenCV's MOG2 model learns an adaptive background. Shadow-valued pixels are removed by a high binary threshold, followed by opening and dilation. After four warm-up frames, contours between 500 pixels and 65% of the frame area become moving regions. A lightweight centroid tracker associates each current box with the nearest existing centre, assigns a new ID when required and removes IDs after eight missed frames. This method is fast and easy to explain, although it can exchange identities when objects cross.

Input validation and robustness controls

- Rejects null, empty, malformed and smaller-than-32-pixel inputs with readable errors.
- Handles grayscale and four-channel images by converting them to three-channel BGR.
- Resizes wide inputs while preserving aspect ratio to bound latency and memory use.
- Returns a clear no-marker message for blank or unmatched scenes instead of failing.
- Uses MP4 output with an AVI fallback if the local codec cannot initialise.

Custom data and model training

Dataset construction

The notebook generates 560 labelled images, with 140 examples in each of four classes: stop, warning, safe and other. Variations include background level and gradient, scale, rotation, brightness, blur, Gaussian noise and partial occlusion. The other class is a red square with a diagonal line, intentionally designed to pass the colour proposal stage and force the recogniser to use shape evidence. Random seeds are fixed for reproducibility.

Custom synthetic dataset samples

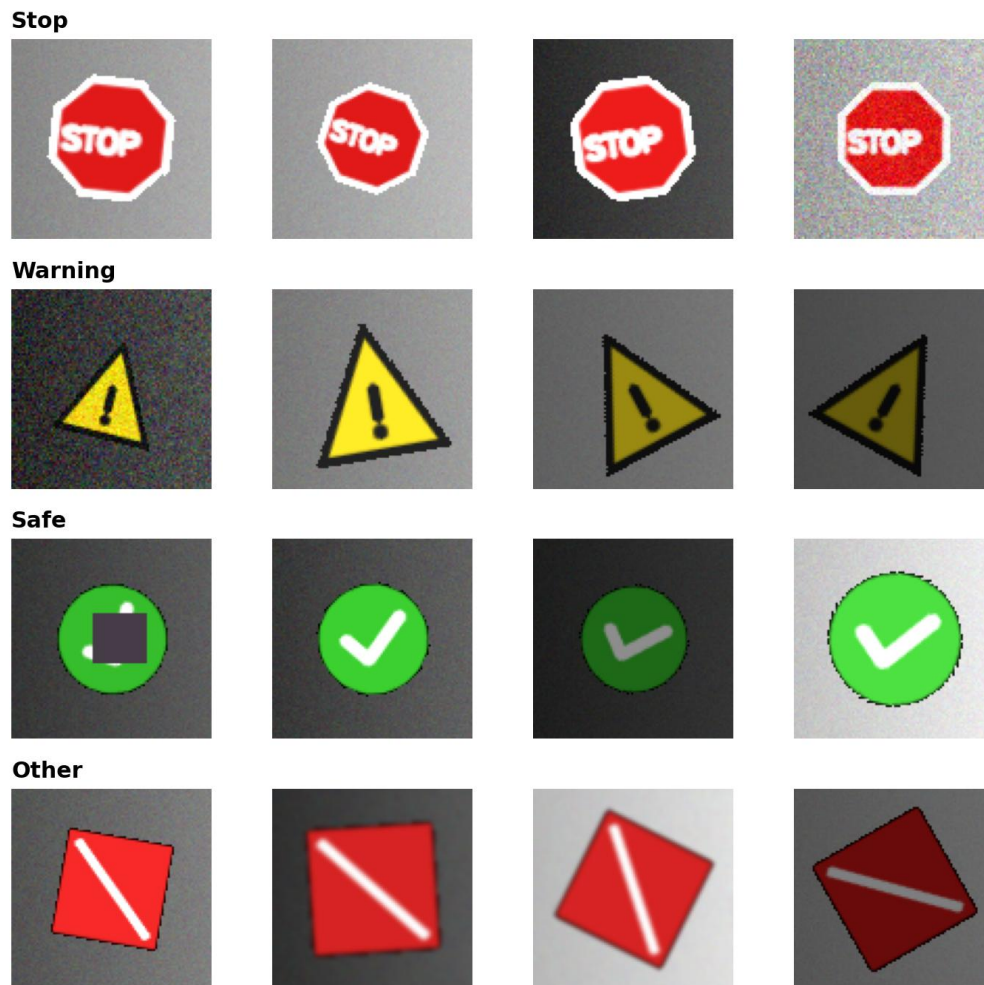


Figure 3. Examples from the custom synthetic dataset.

Features and candidate models

Each canonical crop is represented by HOG gradients, 18-bin hue and 12-bin saturation histograms, four intensity/colour statistics and seven log-transformed Hu moments. We used a stratified 75/25 split and compared three lightweight classifiers on the same samples. Scaling is included inside the SVM and k-NN pipelines to avoid information leakage from the held-out split.

Model	Validation accuracy	Training (s)	Inference (ms/crop)
SVM (RBF)	100.0%	0.100	0.078
Random forest	100.0%	0.188	0.168
k-NN	100.0%	0.004	0.363

Table 1. Candidate-model comparison from the executed notebook.

Model choice

All three models separated the controlled crop dataset perfectly, so crop accuracy alone could not justify a choice. We selected the RBF SVM because it combined a non-linear boundary, class balancing, probability output for user-facing confidence and low per-crop inference time. The trained pipeline is saved as a joblib artifact and is also recreated from source whenever the notebook is run.

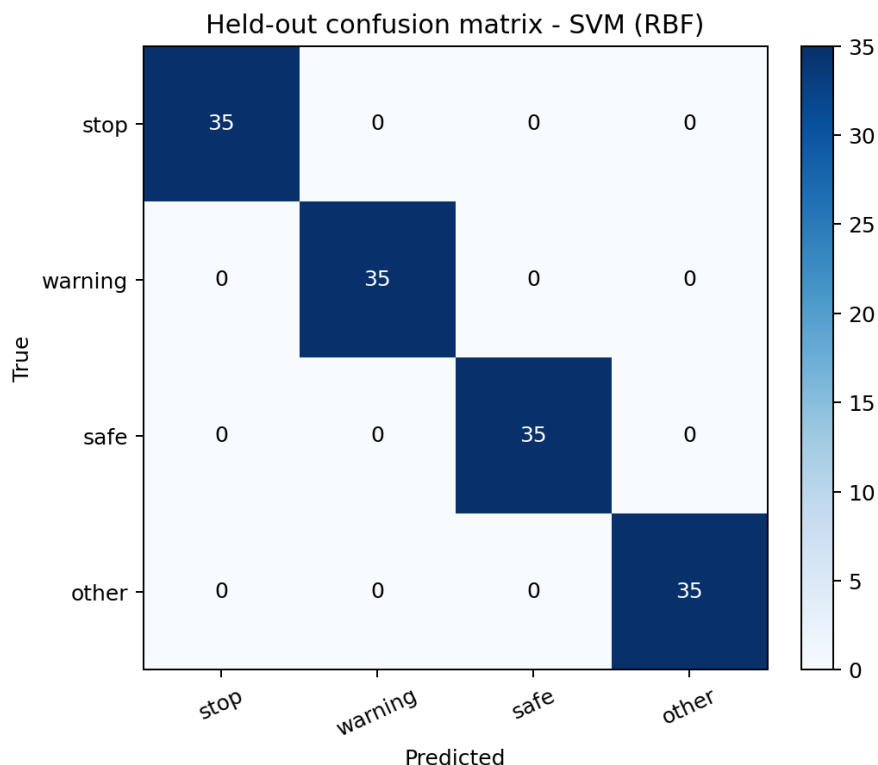


Figure 4. Held-out confusion matrix for the selected SVM.

Interpretation warning. The 100% held-out crop accuracy applies only to the generated marker distribution. It must not be presented as real-world safety accuracy. End-to-end scene metrics are more demanding and are reported separately.

Experiments and results

Evaluation protocol

Recognition robustness used 35 unseen crops per class and condition. End-to-end detection used 12 scenes per condition, with one to three target classes per scene and seeds not used for training. A prediction counted as a true positive only when its class matched and its intersection-over-union with an unused ground-truth box was at least 0.30. Precision, recall, F1 and mean matched IoU were then aggregated. The seven scene conditions were bright, dim, rotated, blurred, noisy, occluded and cluttered.

Condition	Precision	Recall	F1	Mean IoU	Recognition accuracy
Bright	100.0%	95.8%	97.9%	0.762	100.0%
Dim	95.2%	83.3%	88.9%	0.725	100.0%
Rotated	90.0%	75.0%	81.8%	0.774	94.3%
Blurred	100.0%	100.0%	100.0%	0.795	100.0%
Noisy	100.0%	100.0%	100.0%	0.757	100.0%
Occluded	94.7%	75.0%	83.7%	0.714	100.0%
Cluttered	67.7%	87.5%	76.4%	0.745	N/A

Table 2. Condition-wise robustness results from the executed notebook.

Across the seven conditions, mean precision was 92.5%, mean recall was 88.1% and mean F1 was 89.8%. Blurred and noisy generated scenes reached perfect detection scores in this run. Rotated, occluded and cluttered scenes were harder. Clutter reduced precision because saturated distractors sometimes survived proposal filtering; rotated and occluded targets mainly reduced recall. This pattern is consistent with the proposal/classification design.

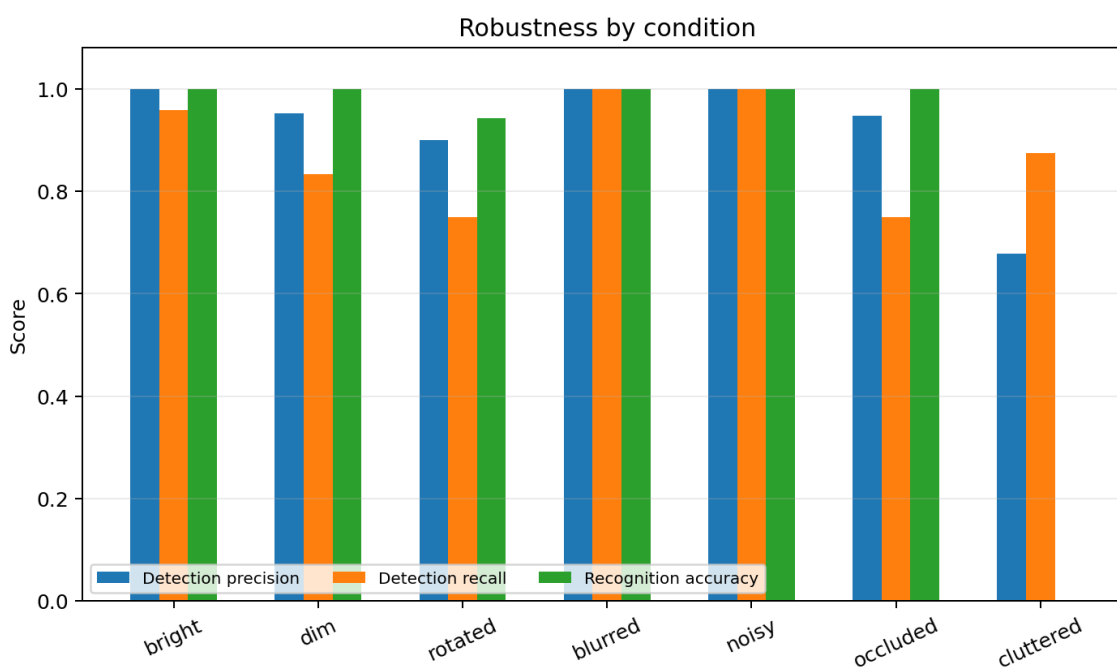


Figure 5. Detection and recognition robustness by condition.

Performance

Measure	Result
Still-frame mean latency	13.47 ms
Still-frame median latency	13.57 ms
Still-frame 95th-percentile latency	15.08 ms

Measure	Result
Estimated still-frame throughput	74.23 FPS
Processed-video mean latency	13.27 ms/frame
Processed-video estimated throughput	75.36 FPS
Rubric threshold	10 FPS - met

Table 3. CPU timing measured over the complete integrated pipeline.

The benchmark includes validation, resizing, enhancement, edges, proposal generation, feature extraction, SVM inference, drawing and guidance. Video timing also includes MOG2 and tracking. FPS is computed as the reciprocal of processing latency and therefore measures processing capacity rather than camera-display refresh rate. Results depend on hardware, but the tested run had a substantial margin above the 10 FPS criterion.

Qualitative results

Safe-route marker on the right (97% confidence); Warning marker on the centre (95% confidence); Stop marker on the left (57% confidence).

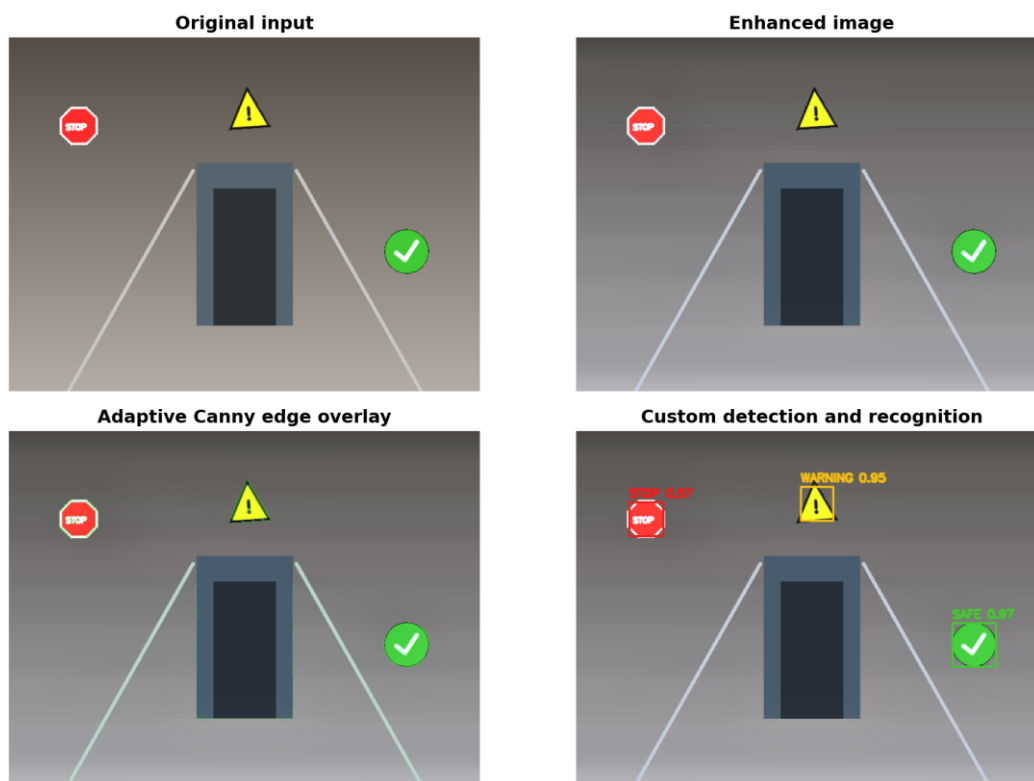


Figure 6. End-to-end image workflow and generated guidance.

The clean defence scene demonstrates how the tasks support one another. Enhancement normalises contrast, the edge view exposes structural boundaries, colour/contour proposals localise candidates, the SVM labels all three markers and the fusion layer produces: 'Safe-route marker on the right; warning marker in the centre; stop marker on the left.' Video examples add motion boxes and persistent IDs using the same base pipeline.

Correctness and edge-case tests

- Blank uniform scene: zero marker detections and a readable no-marker message.
- Tiny 12 x 12 image: rejected with a ValueError instead of entering OpenCV operations.
- Dim, rotated, blurred, noisy and occluded scenes: processed and scored condition by condition.
- Cluttered scene: false positives are measured rather than removed from the reported results.
- Generated 90-frame video: successfully encoded, processed and measured with motion/tracking outputs.

Challenges and critical analysis

Training-to-inference mismatch

An early version classified padded scene crops directly even though training used tightly framed 96 x 96 examples. Crop accuracy looked perfect, but scene recall was only 7.1%. This was a useful warning against reporting a convenient metric without testing the complete system. Canonical foreground scaling and recognition from pre-enhancement pixels raised the verified mean scene F1 to the value reported in Table 2.

Remaining limitations

- Synthetic-to-real domain gap: natural signs, reflections, weathering and camera compression are not represented.
- Colour dependence: extreme colour casts or desaturation may prevent proposal generation before the SVM runs.
- No calibrated distance: left/centre/right guidance is relative image position, not metric depth or collision risk.
- Motion is not semantic: MOG2 identifies changing regions but cannot state whether they are people or vehicles.
- Simple tracking: nearest-centroid matching may exchange IDs during crossing or prolonged occlusion.
- Safety and accessibility: no target-user study, audio/haptic interface or formal false-negative safety threshold was completed.

UI and deployment trade-offs

Gradio made the application fast to reproduce and suitable for a live local defence. It also exposes upload and webcam sources without maintaining a separate JavaScript codebase. The trade-off is less control over mobile layout and browser codec behaviour than a custom frontend. Local hosting avoids sending camera data to a cloud service, but the prototype does not yet implement persistent privacy controls or authentication.

Individual contributions

The following table records the agreed primary workstreams. Integration, review and defence preparation were shared so that each member contributes to coding, documentation and presentation as required.

Member	Student ID	Primary coding	Documentation	Defence role
Mehdi Leghmizi	8528834	UI integration and input handling	User story and requirements	Opening and UI demo
Neeraj Santosh	8329345	Custom dataset and model training	Training methodology	Model selection
Muhammad Soban	8555588	Enhancement, edges and robustness tests	Experiment methodology	Image robustness
Zachary Bracke	8947405	Motion, tracking and performance	Results and critical analysis	Video and performance
Mostafa Shalash	7391493	System integration and QA	Architecture, conclusion and references	Architecture and closing

Table 4. Group contribution record and defence responsibilities.

Conclusion and future work

The project demonstrates a working integrated system rather than a collection of isolated scripts. Six permissible computer vision capabilities operate inside one image/video workflow and are exposed through a responsive local web interface. The custom SVM satisfies the training requirement, the end-to-end evaluation tests robustness rather than relying only on crop accuracy, and the measured performance is comfortably above the real-time threshold. The most important lesson was that complete-system evaluation can reveal failures hidden by excellent component metrics.

Future work should collect consented real images and videos, annotate objects with bounding boxes, compare a fine-tuned detector with the current proposal-plus-classifier approach, calibrate depth, quantify false negatives by hazard severity and test audio/haptic guidance with target users. A mobile deployment would also need model compression, battery profiling, offline inference and privacy-by-design review.

References

- Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools.
- Canny, J. (1986). A computational approach to edge detection. IEEE Transactions on Pattern Analysis and Machine Intelligence, 8(6), 679-698. <https://doi.org/10.1109/TPAMI.1986.4767851>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20, 273-297. <https://doi.org/10.1007/BF00994018>
- Dalal, N., & Triggs, B. (2005). Histograms of oriented gradients for human detection. Proceedings of CVPR, 886-893. <https://doi.org/10.1109/CVPR.2005.177>
- Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

Zivkovic, Z. (2004). Improved adaptive Gaussian mixture model for background subtraction. Proceedings of ICPR, 28-31. <https://doi.org/10.1109/ICPR.2004.1333992>

OpenCV. (2026). OpenCV documentation. <https://docs.opencv.org/4.x/>

Scikit-learn developers. (2026). SVC documentation. <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>

Gradio. (2026). Gradio documentation. <https://www.gradio.app/docs>

University of Wollongong in Dubai. (2026). CSCI435 project description, Spring 2026.

Appendix A - Rubric traceability

Criterion	Marks	Evidence
Four or more tasks	15	Six integrated capabilities in VisionSystem; Figures 1 and 6.
Correctness and robustness	15	Seven conditions, edge-case assertions and class-aware IoU evaluation; Table 2.
UI and UX	10	Verified Gradio image/webcam and video tabs; Figure 2.
Model selection and training	10	560 custom images, three-model comparison and saved SVM; Table 1.
Architecture and code quality	10	Typed functions/classes, unified notebook, README and architecture diagram.
Performance	5	74.2 FPS measured versus 10 FPS requirement; Table 3.
Report quality	15	Required sections, diagrams, quantitative results, qualitative examples and critical analysis.
Live demonstration	15	Local interface, stable generated examples, 2:05 backup video and defence guide.
Teamwork	5	Named coding, documentation and defence responsibilities; Table 4.

Appendix B - Reproduction and defence checklist

- Create a Python 3.12 virtual environment and install requirements.txt.
- Open CSCI435_Project.ipynb and run all cells from top to bottom.
- Confirm that the model comparison, confusion matrix, robustness table and performance assertion appear.
- Open the printed local Gradio URL and process the generated defence image and sample video.
- Show image enhancement, edge overlay, marker boxes, guidance, motion IDs and measured FPS.
- Keep artifacts/CSCI435_Demonstration_Video.mp4 available as the required video and technical-failure backup.
- Push the repository to GitHub or GitLab, grant the lecturer access and place the repository link in the Moodle submission.